



ONCHAIN CORE BANKING

DEPLOYMENT GUIDE · V1.0 · SEPTEMBER 2026

Two Deployment Backends, One Platform

The same DFNS platform runs on two trust substrates. Baseline is built on self-hosted HashiCorp Vault; Enterprise AWS-Native is built on AWS Secrets Manager, KMS and IAM. This one-pager maps each managed service to its self-hosted counterpart, so a technical team can see exactly which infrastructure components it must provide in its own environment.

SECRETS STORE

SERVICE IDENTITY

PKI / MTLS

KAFKA AUTH

DATABASE AUTH

MPC SIGNING

IAM EVERYWHERE

SINGLE-TENANT

CLIENT-OPERATED

01 Same application, different substrate

Both deployment types run the **same DFNS platform**: the identical set of container images, the same MPC signing model, and the same data model. DFNS is an Onchain Core Banking platform, and its signing layer uses an MPC threshold scheme in which each private key is split into shares held by separate signer nodes, so no single node ever holds a whole key. What differs between the two backends is the layer underneath: how secrets, service identity, PKI and the managed data services are provided. The platform is built against pluggable backends, so each AWS-managed service has a self-hosted counterpart, and each self-hosted service has an AWS-managed counterpart.

THREE DFNS-SPECIFIC TERMS, IN BRIEF

MPC signing model	A coordinator drives a set of signer nodes to co-produce a signature from their individual key shares, without ever reconstructing the whole key in one place.
Keyshares store	The datastore that holds each signer's MPC key shares, kept alongside the per-service databases.
External Secrets Operator	A Kubernetes controller that reads secrets from an external store (here, AWS Secrets Manager) and materializes them as native Kubernetes Secrets inside the cluster.

SUMMARY

Baseline is self-hosted trust, built on HashiCorp Vault. Enterprise AWS-Native is AWS-managed trust, built on Secrets Manager, KMS and IAM. There is no Vault in the Enterprise AWS-Native type; services authenticate to their dependencies with short-lived IAM credentials instead of static passwords.

02 Component-by-component contrast

CAPABILITY	BASELINE self-hosted trust	ENTERPRISE AWS-NATIVE AWS-managed trust
Secrets store	HashiCorp Vault (KV)	AWS Secrets Manager
Envelope encryption and KMS	Vault Transit	AWS KMS
Service identity	Vault Kubernetes auth (one role per service)	AWS IAM (IRSA or Pod Identity)
Secrets delivery into the cluster	Vault Agent injector	External Secrets Operator
PKI and mTLS	Vault PKI	cert-manager with ACME, plus Istio mesh
Kafka authentication	SCRAM (password)	IAM (<code>aws-msk-iam</code> , passwordless)
Database authentication	PostgreSQL password	RDS or Aurora IAM auth, fail-closed TLS
Cache authentication	Redis password	ElastiCache IAM
Eventing	Kafka	Kafka only (no SQS, SNS or DynamoDB)
Hosting	Single-tenant	Single-tenant, in the client's own AWS account
Minimum platform release	≥ 1.929	≥ 1.935

Both profiles are single-tenant and run the identical platform images. Only the trust and infrastructure substrate in the columns above changes; the application and signing planes do not.

03 What is identical across both types

- ◆ The **container images** and how they expand into pods: the same set of services, APIs and cron workers in both types.
- ◆ The **MPC signing model**: coordinator, signer nodes and delivery, with the same signing group and threshold configuration.
- ◆ The **service dependency order** and the initialization jobs (database and platform bootstrap).
- ◆ The **data model**, meaning the per-service databases plus the keyshares store that holds the MPC key shares, and the ingress hosts.

04 What changes between them

- 01 **The Vault plane is replaced, not removed piecemeal.** Vault's three roles, meaning secrets storage, envelope encryption and PKI, are covered by AWS Secrets Manager, AWS KMS and cert-manager respectively, with the External Secrets Operator delivering secrets into the cluster.
- 02 **Identity becomes IAM everywhere.** Every service call to Kafka, the cache, the databases, Secrets Manager or KMS uses a passwordless, short-lived IAM credential; static passwords are eliminated across the board.
- 03 **Eventing simplifies to Kafka only.** The SQS, SNS and DynamoDB paths used in the DFNS-hosted SaaS are switched off in favor of Kafka and PostgreSQL.
- 04 **The platform runs in the client's own AWS account.** DFNS supplies the images, charts and configuration; the client operates the infrastructure.

05 Deployment notes

DEFAULT PROFILE

The deployment kit defaults to the Baseline profile (Vault plus SCRAM). The AWS-Native profile is enabled through explicit configuration overrides and requires the `≥ 1.935` platform release.

- ◆ A **self-evaluation checklist** accompanies this document for validating an AWS-Native deployment end to end in the target environment: IAM connectivity to Kafka, cache and database; External Secrets Operator sync; and certificate issuance.
- ◆ In the **architecture diagrams**, the application plane and the signing plane are the same in both types. The secrets and identity plane and the secret-bootstrap flow are the parts that differ, and they are drawn per type.